```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#Step 1: Data Loading and Initial Explora

#Load all datasets using Pandas
customers_url ="/Users/cegemba/sandbox/Lo
location_url ="/Users/cegemba/sandbox/Log
order_items_url = "/Users/cegemba/sandbox
orders_url = "/Users/cegemba/sandbox/Log
payments_url = "/Users/cegemba/sandbox/Lo
products_url = "/Users/cegemba/sandbox/Lo
reviews_url = "/Users/cegemba/sandbox/Log
sellers_url = "/Users/cegemba/sandbox/Log
category_translation_url = "/Users/cegemb

#Inspect structure using .head(), .info()
customers = pd.read_csv(customers_url)
location = pd.read_csv(location_url)
order_items = pd.read_csv(order_items_url
orders = pd.read_csv(orders_url)
payments = pd.read_csv(payments_url)
products = pd.read_csv(products_url)
reviews = pd.read_csv(reviews_url)
sellers = pd.read_csv(sellers_url )
category_translation = pd.read_csv(catego


tables = {
    'orders': orders,
    'order_items': order_items,
    'payments': payments,
    'reviews': reviews,
    'sellers': sellers,
    'category_translation': category_tra
    'location': location
```

```
    'location': location,
    'products': products,
    'location': location,
    'customers': customers
}
for name, table in tables.items():
    print(f"----------{name} de
    print(table.info())
    print(table.head(2))
    print(table.describe())
    print(f"----------{name}


#Step 2: Data Cleaning and Preprocessing

#Handle missing values appropriately
customers.isnull().sum()
location.isnull().sum()
order_items.isnull().sum()

#Order table has somes null values
orders.isnull().sum()

payments.isnull().sum()

products.isnull().sum()
#Fix products null values
products['product_category_name'] = produ
products['product_description_lenght'] =
products['product_name_lenght'] = product
products['product_photos_qty'] = products
products['product_weight_g'] = products[
products['product_length_cm'] = products
products['product_height_cm'] = products
products['product_width_cm'] = products[

reviews.isnull().sum()
#Fix reviews null values
reviews['review_comment_title'] = reviews
reviews['review_comment_message'] = revie
```

```
sellers.isnull().sum()
category_translation.isnull().sum()

#Remove duplicate records
customers.duplicated().sum()

#Fix duplicates location
location.duplicated().sum()
location = location.drop_duplicates()

order_items.duplicated().sum()
orders['order_id'].duplicated().sum()
payments.duplicated().sum()
reviews.duplicated().sum()
sellers.duplicated().sum()
category_translation.duplicated().sum()

#Convert date columns to datetime format
#Converts date colums in orders table to
cols = ['order_delivered_carrier_date',
for col in cols:
    orders[col] = pd.to_datetime(orders[c

#Converts date colums in order_items tabl
order_items['shipping_limit_date'] = pd.t

#Converts date colums in reviews table to
reviews_cols_date = ['review_creation_dat
for col in reviews_cols_date:
    reviews[col] = pd.to_datetime(reviews
print(reviews.info())

#Validate data types and ranges
tables = [orders, order_items, payments,
for table in tables:
    print(table.dtypes)

#Fixing ranges
location = location[location['geolocatio
```

```
#Standardize column names if required
table_cols= [orders, order_items, payment
for tab in table_cols:
    print(tab.columns)

# Step 3: Data Integration (Critical Comp
master = orders.copy()
#orders + customers
master = master.merge(customers, on='cust

#orders + order_items
master = orders.merge(order_items, on='or

#order_items + products
master = master.merge(products, on='produ

#orders + payments
master = master.merge(payments, on='order

#orders + reviews
master = master.merge(reviews, on='order_

#order_items + sellers
master = master.merge(sellers, on='seller

#products + category_translation
master = master.merge(category_translatio

print(master.shape)
print(master.head(2))

# Step 4: Feature Engineering
# Create meaningful features such as:

#Total order value (aggregated from order
order_value = (order_items.groupby('order
master = master.merge(order_value, on='or
```

```
#Delivery time (order purchase to delivel
orders['order_delivery_time'] = orders['o
master = master.merge(orders[['order_id',

#Number of items per order
items_per_order = (order_items.groupby('o
master = master.merge(items_per_order, on

#Customer purchase frequency
ustomer_purchase_frequency = (orders.grou
master = master.merge(customer_purchase_f

#Customer lifetime value (basic approxima
order_level = master[['order_id', 'custom
customer_lifetime_value = (order_level.gr
master = master.merge(customer_lifetime_v

#Average order value per customer
average_order_value = (order_level.groupb
master = master.merge(average_order_value


# Step 5: Exploratory Data Analysis (EDA)
# Perform structured analysis across the

# Customer Analysis
#New vs repeat customers
customer_summary = (master[['customer_id
customer_summary['customer_type'] = custo

#High-value vs low-value customers
customer_summary = (master[['customer_id
threshold = customer_summary['customer_li

customer_summary['value_segement'] = cust
    lambda x: 'High Value' if x >= thresh
)
print(customer_summary['value_segement'].

#Geographic distribution of customers
```

```
customer_geo = (master[['customer_id', 'c
geo_distribution = (customer_geo.groupby(
print(geo_distribution)

# Revenue and Order Analysis
# Monthly revenue trends
order_level['order_month'] = order_level
monthly_revenue = (
    order_level
    .groupby('order_month')
    .agg(monthly_revenue=('total_order_va
    .reset_index()
    )
print(monthly_revenue)

#Order volume trends
monthly_orders = (
    order_level
    .groupby('order_month')
    .agg(order_volume=('order_id', 'nunic
    .reset_index()
)
print(monthly_orders.head(3))

#Peak sales periods
peak_sales = (monthly_revenue.sort_values
print(peak_sales.head(3))

# Product Analysis
#Top-selling product categories
top_categories =(
    master
    .groupby('product_category_name_engli
    .agg(items_sold= ('order_item_id', 'c
    .reset_index()
    .sort_values('items_sold', ascending=
)
print(top_categories.head())
```

```
# Revenue contribution by category
category_revenue = (
    master
    .groupby('product_category_name_engli
    .agg(category_revenue=('price', 'sum
    .reset_index()
    .sort_values('category_revenue', asce
)
print(category_revenue)

#Product demand distribution
product_demand = (
    master
    .groupby('product_id')
    .agg(order_count=('order_id', 'nuniqu
    .reset_index()
    .sort_values('order_count', ascending
)
print(product_demand.head(10))

# Seller Analysis
#Top-performing sellers
top_sellers = (
    master
    .groupby('seller_id')
    .agg(
        seller_revenue=('price', 'sum'),
        orders=('order_id', 'nunique'),
        items_sold=('order_item_id', 'cou
    )
    .reset_index()
    .sort_values('seller_revenue', ascend
)
print(top_sellers)

#Seller contribution to revenue
seller_revenue = (
    master
    .groupby('seller_id')
    .agg(seller_revenue=('price', 'sum'))
```

```
    .reset_index()
)
total_revenue = seller_revenue['seller_re

seller_revenue['revenue_percent'] = (
    seller_revenue['seller_revenue']/tota
)
seller_revenue = seller_revenue.sort_valu
print(seller_revenue.head())

#Seller distribution
seller_distribution = (
    master[['seller_id', 'seller_state']]
    .drop_duplicates()
    .groupby('seller_state')
    .size()
    .reset_index(name='num_sellers')
    .sort_values('num_sellers', ascending
)
print(seller_distribution)

# Review and Satisfaction Analysis
#Distribution of review scores
review_distribution = (
    reviews
    .groupby('review_score')
    .size()
    .reset_index(name='count')
    .sort_values('review_score')
)
print(review_distribution)

#Relationship between delivery time and r
review_delivery = (
    reviews[['order_id', 'review_score']]
    .merge(
        orders[[
            'order_id',
            'order_purchase_timestamp',
            'order_delivered_customer_da
```

```
        ]],
        on='order_id',
        how='left'
    )
    .copy()
)


review_delivery['delivery_days'] = (
    review_delivery['order_delivered_cust
    - review_delivery['order_purchase_tir
).dt.days

deliver_vs_rating = (
    review_delivery
    .groupby('review_score')
    .agg(avg_delivery_days=('delivery_day
    .reset_index()
    .sort_values('review_score')
)
print(deliver_vs_rating)

#Identification of dissatisfaction patter
low_reviews = review_delivery[
    review_delivery['review_score'] <=2
    ]

low_review_summary = (
    low_reviews
    .agg(
        avg_delivery_days=('delivery_days
        num_reviews=('review_score', 'cou
    )
)

review_groups = review_delivery.copy()

review_groups['review_group']= review_gro
    lambda x: 'low (1-2)' if x <= 2 else
```

```
)

dissatisfaction_pattern = (
    review_groups
    .groupby('review_group')
    .agg(
        avg_delivery_days=('delivery_days
        num_reviews=('review_score', 'cou
    )
    .reset_index()
)
print(dissatisfaction_pattern)

# Step 6: Data Visualization
# Use Matplotlib and Seaborn to create:
#Time series plots (sales trends)
plt.figure(figsize=(12, 5))
plt.plot(
    monthly_revenue_plot['order_month'],
    monthly_revenue_plot['monthly_revenue
    marker='o'
)

plt.title('Monthly Revenue Trend')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

#Bar charts (category performance)
plt.figure(figsize=(12, 6))
sns.barplot(
    data = top_category_plot,
    x='category_revenue',
    y='product_category_name_english'
)

plt.title('Top Category Performance')
plt.xlabel('Revenue')
```

```
plt.ylabel('Product Category')
plt.tight_layout()
plt.show()

#Histograms (distribution analysis)
plt.figure(figsize=(10, 5))
sns.histplot(
    data=review_delivery,
    x='delivery_days',
    bins=30
)

plt.title('Distribution of Delivery Days
plt.xlabel('Delivery Days')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()


#Box plots (outlier detection)
plt.figure(figsize=(10, 5))
sns.boxplot(
    data=review_delivery,
    x='review_score',
    y='delivery_days'
)
plt.title('Delivery Days Distribution by
plt.xlabel('Review Score')
plt.ylabel('Delievry Days')

plt.tight_layout()
plt.show()

#Heatmaps (correlation analysis)
corr_data = review_delivery[['review_sco

plt.figure(figsize=(6, 4))

sns.heatmap(
```

```
    corr_data,
    annot=True,
    cmap='Blues'
)
plt.title('Correlation Heatmap')


plt.tight_layout()
plt.show()

# Step 7: Business Insights and Recommend
# You must derive clear and actionable in

# Identify top revenue-driving factors
#Top revenue driving are Catergory revenu
print(category_revenue.head(10))
print(seller_revenue.head(10))

# Highlight customer behavior patterns
#Repeat customers generated higher value
print(customer_summary['value_segement'].

# Evaluate operational inefficiencies
#The analysis reveal that orders with lon
#experienced delayed deliveries were more
print(dissatisfaction_pattern)
print(deliver_vs_rating)

#Provide strategic recommendations
# Insights must be supported by data and

#Optimize shipping processes, reduce dela
#Focus on high performing Categories, pri
```

```
----------------Cusomers details-------
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 5 columns):
 #   Column                    Non-Null
```

```
---  ------                        --------
 0   customer_id                   99441 nc
 1   customer_unique_id            99441 nc
 2   customer_zip_code_prefix      99441 nc
 3   customer_city                 99441 nc
 4   customer_state                99441 nc
dtypes: int64(1), object(4)
memory usage: 3.8+ MB
                          customer_id
0  06b8999e2fba1a1fbc88172c00ba8bc7  8(
1  18955e83d337fd6b2def6b18a428ac77  2(
2  4e7b3e00288586ebd08712fdd0374a03  0(

   customer_zip_code_prefix          cu
0                     14409
1                      9790  sao bernai
2                      1151
----------------Location details-------
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000163 entries, 0 to 1000
Data columns (total 5 columns):
 #   Column                        Non-N
---  ------                        -----
 0   geolocation_zip_code_prefix   1000
 1   geolocation_lat               1000
 2   geolocation_lng               1000
 3   geolocation_city              1000
 4   geolocation_state             1000
dtypes: float64(2), int64(1), object(2)
memory usage: 38.2+ MB
   geolocation_zip_code_prefix  geoloca
0                         1037       -
1                         1046       -
2                         1046       -

  geolocation_city geolocation_state
0        sao paulo                SP
1        sao paulo                SP
2        sao paulo                SP
----------------Order items details----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 112650 entries, 0 to 11264
Data columns (total 7 columns):
```

```
 #   Column                Non-Null Cour
---  ------                --------------
 0   order_id              112650 non-nu
 1   order_item_id         112650 non-nu
 2   product_id            112650 non-nu
 3   seller_id             112650 non-nu
 4   shipping_limit_date   112650 non-nu
 5   price                 112650 non-nu
 6   freight_value         112650 non-nu
```

Start coding or generate with AI.